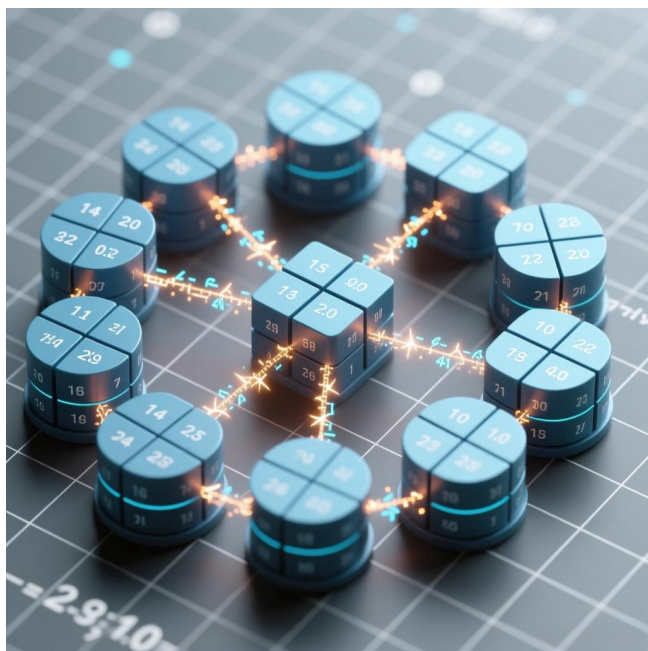




王晓茹



北京邮电大学





目录

CONTENT

01 | 动态规划核心思想

02 | 矩阵连乘问题建模

03 | 分治递归与指数瓶颈

04 | 自顶向下备忘录

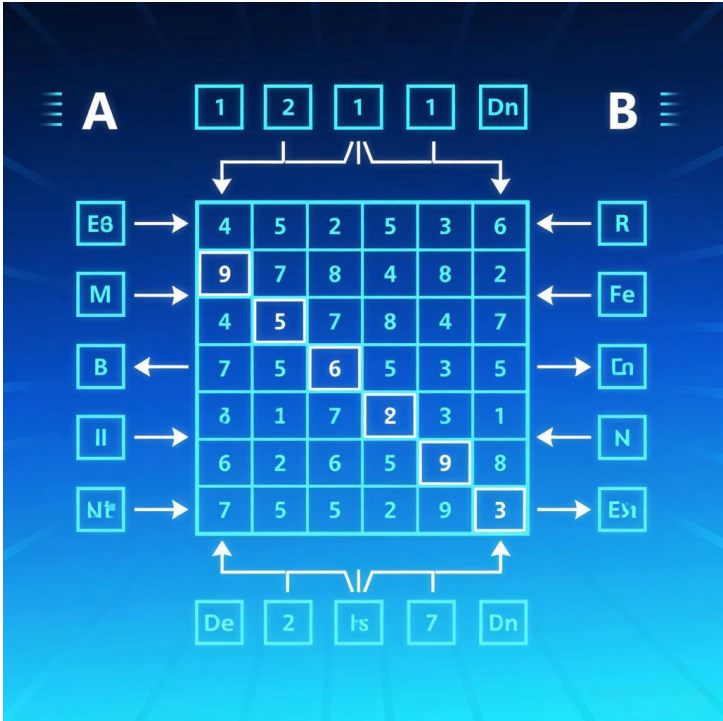
05 | 自底向上动态规划

06 | 三方法对比与选型





PART 01



动态规划核心思想



动态规划的定义与历史脉络



理查德·贝尔曼

Richard Bellman

20世纪50年代

由美国数学家理查德·贝尔曼提出，是运筹学与计算机科学领域的重要里程碑。

从暴力到聪明的遍历

动态规划的诞生

动态规划是一种将复杂问题分解为简单子问题的算法设计范式，通过 **保存子问题的解** 来避免重复计算，核心在于“用空间换时间”。



避免重复计算

存储已解决的子问题答案，
需要时直接调用。



逐步构建最优解

从最小子问题开始，逐步
解决更大规模的问题。



提升算法效率

将指数级时间复杂度降至
多项式级别。

重叠子问题：指数爆炸的元凶

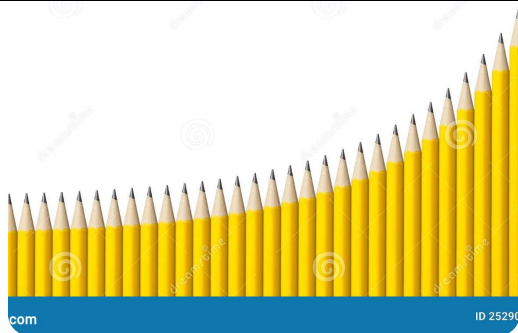
在递归求解过程中，相同的子问题被反复计算，导致时间复杂度呈指数级膨胀。

$$\begin{cases} F(n) = F(n-1) + F(n-2) & n \geq 2 \\ F(0) = 0, F(1) = 1 \end{cases}$$

朴素递归

斐波那契数列的递归函数

```
1: fib(n)
2:   if n <= 1
3:     then return n
4:   else return
```



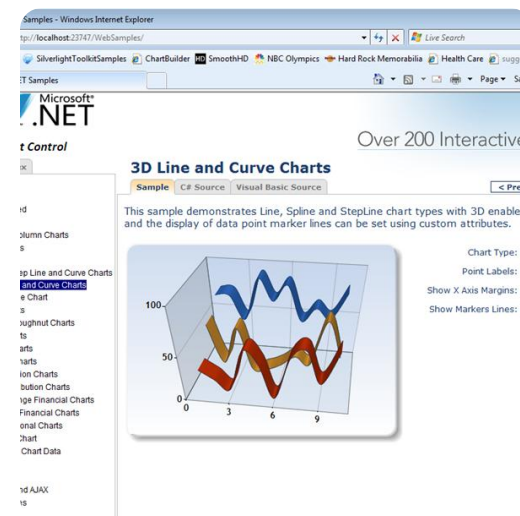
以斐波那契数列为例， $F(3)$, $F(2)$ 等子问题被大量重复计算。

时间复杂度: $O(2^n)$



记忆化优化

动态规划

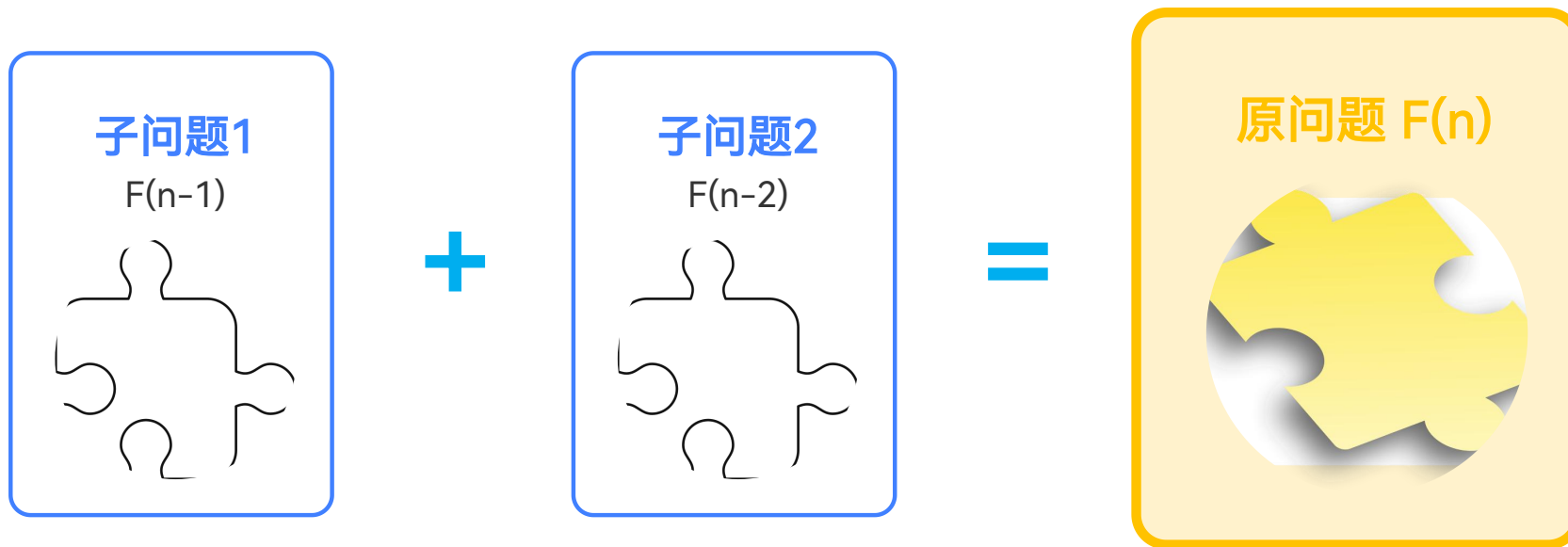


通过记录子问题答案，每个子问题只需计算一次。

时间复杂度: $O(n)$

最优子结构：全局最优如何由局部最优拼成

原问题的最优解包含了其子问题的最优解。



大问题的最优解可以通过小问题的最优解“拼接”组合得出，且这一过程具有无后效性。

无后效性：决策的独立时间轴

过去 (已解决子问题)

子问题一旦求解完成，其答案便被记录，如同历史被写入史册，不可篡改。



现在 (当前决策)

当前状态的选择，只依赖于已记录的过去，不会影响之前的任何状态。



未来 (后续子问题)

未来的求解过程，只依赖于当前状态的结果，形成一条清晰、独立的决策链。



动态规划的适用条件



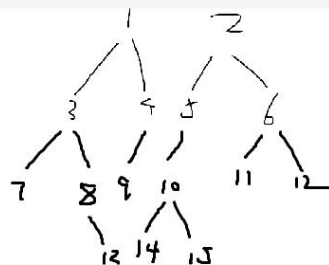
最优子结构

问题的最优解包含其子问题的最优解。例如，矩阵链 $(A_1(A_2A_3))$ 最优，则其子链 (A_2A_3) 也必须最优。



重叠子问题

递归求解时，相同的子问题会被反复计算。如图中 $m[2,3]$ 被多次调用，造成资源浪费。



遍历先右后左，垃圾



无后效性

当前状态确定后，后续决策不会影响之前的状态。选择分割点 k 后，只需考虑 $A[i,k]$ 和 $A[k+1,j]$ 的最优解。

两种实现范式对比

自底向上 (Bottom-up)

思想: 从最小子问题开始，逐步构建更大问题的解。

实现: 通常使用迭代和循环，预先确定计算顺序。
 $root1, memo = \{\}$

优点: 无递归开销，效率稳定，易于进行空间优化。

缺点: 需要仔细分析计算顺序，代码有时不够直观。

斐波那契的“自底向上递推方法”实现

```
1: function bottomUpFibonacci(n)
2:   if n = 0 : return 0
3:   if n = 1 : return 1
4:   // 初始化一个数组来存储中间结果
5:   fib = array of size n+1
6:   fib[0] = 0
7:   fib[1] = 1
8:   // 从2开始迭代计算到n
9:   for i = 2 to n :
10:     fib[i] = fib[i-1] + fib[i-2]
11:   return fib[n]
```

核心思想: 迭代循环 + DP表 = 递推法

自顶向下 (Top-down)

思想: 从原问题开始递归，使用备忘录记录已计算子问题。

实现: 基于递归，按需计算子问题。

优点: 思路自然，代码更接近问题定义，易于理解。

缺点: 存在递归开销，有栈溢出风险，效率可能略低。

斐波那契数列的“记忆化搜索”实现

```
1: fib (n, memo={})
2:   if n in memo
3:     then return memo[n]
4:   if n <= 1
5:     then return n
6:   memo[n] ← fib (n-1, memo) + fib (n-2, memo)
```

核心思想: 递归 + 缓存 = 记忆化搜索

在上述代码中，每个斐波那契数列只计算一次，所以时间复杂度为 $O(n)$ 。

设计四步与优化策略

设计步骤(一): 状态定义

将口语化的问题描述，转化为一个或多个可索引的符号，如 $dp[i][w]$ 。这是整个DP设计的基石。



决策阶段

通常代表前 i 个物品、前 i 个字符等。



剩余资源

通常代表剩余容量、剩余金额等。



状态定义

如 $dp[i][w]$: 前 i 个物品在容量 w 下的最大价值。

状态定义:

在动态规划中，将问题的某个阶段或某个子问题的解表示为“状态”。状态的选择应能全面反映问题的当前情况，同时便于后续状态的推导。

设计步骤(二): 状态转移方程

描述状态之间如何递推的核心关系式。
它定义了如何从已知状态推导出未知状态的最优解。

以斐波那契为例，对于第 n 个数，根据之前的状态值的决策：

状态转移方程

$$F(n) = F(n-1) + F(n-2)$$

以0-1背包为例，对于第 i 个物品，我们面临“选”或“不选”的决策：

不选：最优值等同于 $dp[i-1][w]$ 。

选：最优值为 $dp[i-1][w-wt[i-1]] + val[i-1]$ 。

最终，我们取这两种决策中的最大值。

状态转移方程

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i-1]] + val[i-1])$$

设计四步与优化策略

设计四步与优化策略

设计步骤(三) & (四): 边界与顺序



边界条件

为DP表中最小规模的问题赋初值，是**递推的起点**。

```
dp[0][w] = 0 // 没有物品，价值为0  
dp[i][0] = 0 // 容量为0，价值为0
```

边界条件：边界条件是动态规划问题的基础，它们指定了最小规模问题的解，通常是显而易见的或者可以直接给出的。

正确设置边界条件对于确保算法的正确性至关重要。



填表顺序

确保在计算一个状态时，它所依赖的所有子问题都已被计算过。

```
for i from 1 to n:  
    for w from 0 to W:
```

表（或数组）存储：动态规划通常使用表格（或数组）来存储中间结果，即各个状态的最优解。这种存储方式不仅加快了查找速度，也便于状态之间的转移计算。

线性DP vs. 非线性DP

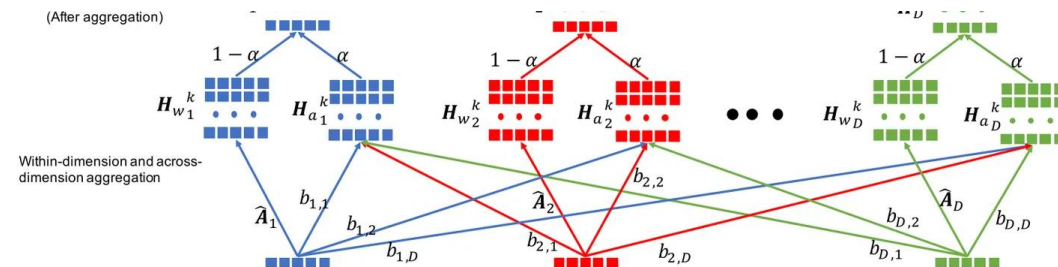
线性动态规划



状态转移是线性的，每个状态仅依赖于有限个前序状态，通常表现为链式或序列式。

示例: 0-1背包, 最长上升子序列(LIS)

非线性动态规划



状态之间的依赖关系复杂，可能形成环状或图状结构，需要特殊的处理顺序。

示例: 最短路径问题, 编辑距离

优化策略：空间优化

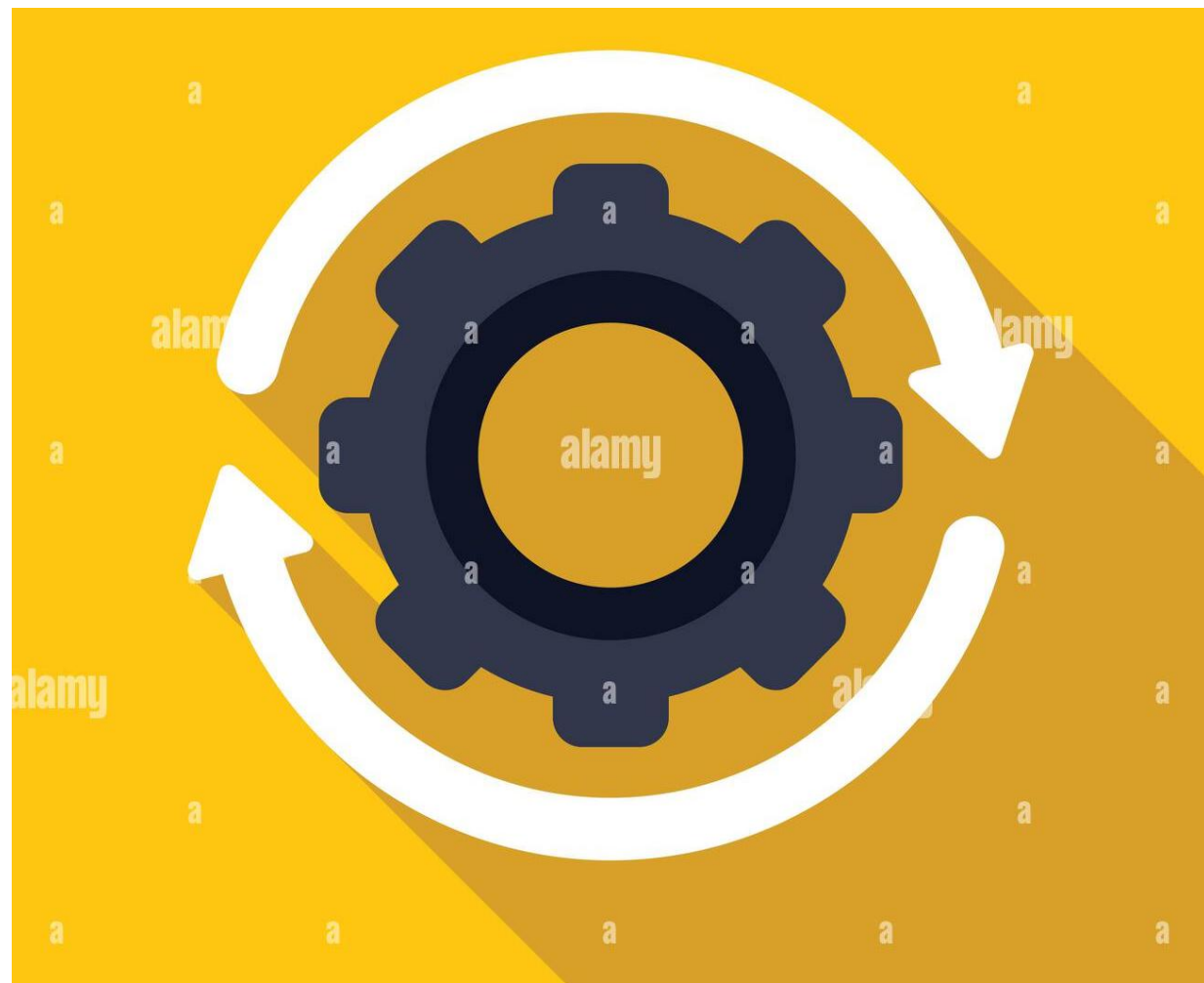
通过观察状态转移方程，我们可以发现很多时候并不需要存储整个DP表，从而大幅降低空间复杂度。

滚动数组

如果状态 i 只依赖于状态 $i-1$ ，则只需保留两行数据，交替使用。

状态压缩

使用位运算等技巧，将集合等复杂状态压缩为整数，节省空间。



优化策略：时间优化

通过数学性质或算法技巧，减少需要枚举的状态或决策点数量。



四边形不等式

利用代价函数的凸性，缩小决策点枚举范围。



决策单调性

最优决策点随区间端点单调移动，可将 $O(n^3)$ 降为 $O(n^2)$ 。



斜率优化

将状态转移视为几何问题，用单调队列维护凸包，优化转移。

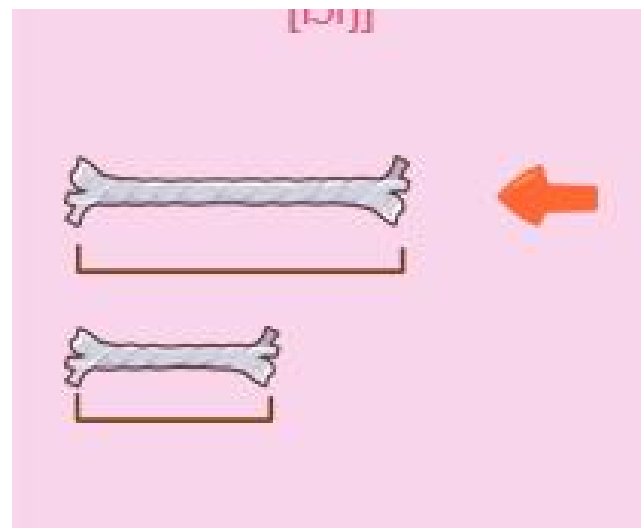


PART 02

区间DP主要针对序列或数组上的子区间优化问题。其核心思想是通过枚举区间内的分割点，将大区间的最优解分解为两个小区间的最优解之和，再加上合并的代价。

通用状态转移方程

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + cost(i, j))$$



矩阵连乘问题建模



矩阵连乘问题描述

1. 问题定义

矩阵链乘法问题可以表述为：给定一系列矩阵 $A_1, A_2, \dots, A_N$ ，其中矩阵 A_i 的维度为 $p_{i-1} \times p_i$ （ $p_0=1, p_n=1$ ），求完全括号化方案，使得这些矩阵连乘的最小标量乘法次数。

2. 完全加括号的矩阵连乘积

- 完全加括号的矩阵连乘积可递归地定义为：
 - 单个矩阵是完全加括号的；
 - 矩阵连乘积A是完全加括号的，则A可表示为2个完全加括号的矩阵连乘积B和C的乘积并加括号，即 $A=(BC)$

不同计算顺序的代价差异巨大。

目标是找到一种括号化方案，使总标量乘法次数最少。

矩阵连乘积 $A_1 A_2 A_3 A_4$ 的不同的加括号方式及其对应计算量分别如下：

$$A_1 = 10 \times 100 \quad A_2 = 100 \times 5 \quad A_3 = 5 \times 50 \quad A_4 = 50 \times 30$$

- (1) $(A_1(A_2(A_3A_4)))$: $5 \times 50 \times 30 + 100 \times 5 \times 30 + 10 \times 100 \times 30 = 52500$
- (2) $(A_1((A_2A_3)A_4))$: $100 \times 5 \times 50 + 100 \times 50 \times 30 + 10 \times 100 \times 30 = 205000$
- (3) $((A_1A_2)(A_3A_4))$: $10 \times 100 \times 5 + 5 \times 50 \times 30 + 10 \times 5 \times 30 = 14000$
- (4) $((A_1(A_2A_3))A_4)$: $100 \times 5 \times 50 + 10 \times 100 \times 50 + 10 \times 50 \times 30 = 90000$
- (5) $((((A_1A_2)A_3)A_4))$: $10 \times 100 \times 5 + 10 \times 5 \times 50 + 10 \times 50 \times 30 = 22500$

矩阵连乘问题：暴力解法

◆直观思考一：穷举法：列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

算法复杂度分析：

对于n个矩阵的连乘积，设其不同的计算次序为P(n)。

由于每种加括号方式都可以分解为两个子矩阵的加括号问题： $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ 可以得到关于P(n)的递推式如下：

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n > 1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

矩阵连乘问题：贪心法

◆直观思考二：计算量小的优先计算

n 个矩阵的连乘积共有 $n-1$ 次乘法计算。首先在 $n-1$ 次乘法计算中选择乘法计算量最小的两个矩阵优先计算；然后再在剩下的 $n-2$ 次乘法计算中选择计算量最小的两个矩阵进行计算，依此往下。

该解决策略在某些情况下可以得到最优解，但是在很多情况下得不到最优解。

如例子的第(5)种完全加括号方式就是采用该策略，但它得到的显然不是最优解。

矩阵连乘积 $A_1 A_2 A_3 A_4$ 的不同的加括号方式及其对应计算量分别如下：

$$A_1 = 10 \times 100 \quad A_2 = 100 \times 5 \quad A_3 = 5 \times 50 \quad A_4 = 50 \times 30$$

- (1) $(A_1(A_2(A_3A_4)))$: $5 \times 50 \times 30 + 100 \times 5 \times 30 + 10 \times 100 \times 30 = 52500$
- (2) $(A_1((A_2A_3)A_4))$: $100 \times 5 \times 50 + 100 \times 50 \times 30 + 10 \times 100 \times 30 = 205000$
- (3) $((A_1A_2)(A_3A_4))$: $10 \times 100 \times 5 + 5 \times 50 \times 30 + 10 \times 5 \times 30 = 14000$
- (4) $((A_1(A_2A_3))A_4)$: $100 \times 5 \times 50 + 10 \times 100 \times 50 + 10 \times 50 \times 30 = 90000$
- (5) $((((A_1A_2)A_3)A_4))$: $10 \times 100 \times 5 + 10 \times 5 \times 50 + 10 \times 50 \times 30 = 22500$

矩阵连乘问题：贪心法

◆直观思考三：矩阵维数大的优先计算

在 n 个矩阵的 $n+1$ 个维数序列 $P_0, P_1, P_2, \dots, P_n$ 中选择维数的最大值(设为 P_i),并把相邻两个含有维数 P_i 的矩阵优先进行计算；然后在剩下的 n 个维数序列中再次选择维数的最大值,进行相应矩阵的乘法运算；依此往下。该解决策略与上一种策略相似,在有些情况下可以得到最优解,但是在有些情况下得不到最优解。如第(3)种完全加括号方式就是采用该策略,显然它得到的是最优解。

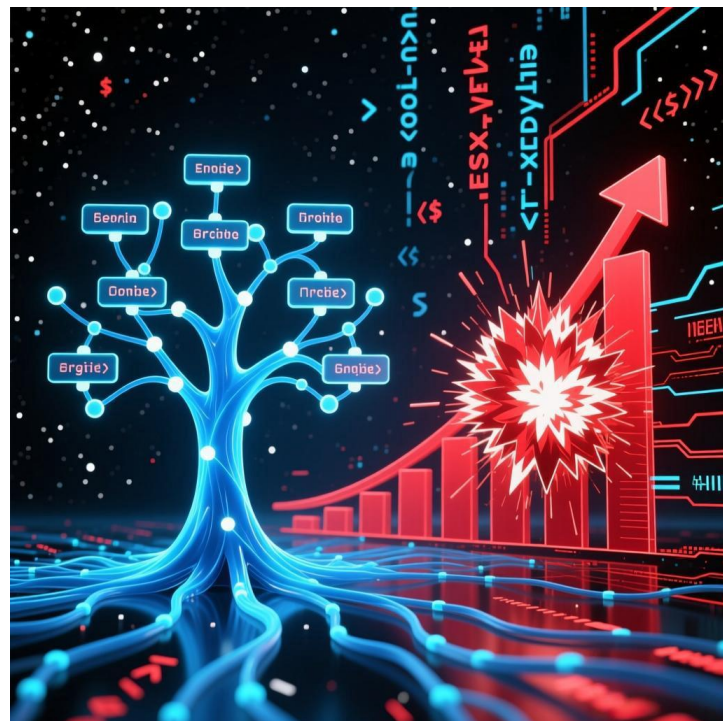
当4个矩阵 A_1, A_2, A_3, A_4 的维数改为

$50 \times 10, 10 \times 40, 40 \times 30$ 和 30×5 时,可以验证,采用该策略得到的就不是最优解。

矩阵连乘积 $A_1 A_2 A_3 A_4$ 的不同的加括号方式及其对应计算量分别如下:

$$A_1 = 10 \times 100 \quad A_2 = 100 \times 5 \quad A_3 = 5 \times 50 \quad A_4 = 50 \times 30$$

- (1) $(A_1(A_2(A_3A_4)))$: $5 \times 50 \times 30 + 100 \times 5 \times 30 + 10 \times 100 \times 30 = 52500$
- (2) $(A_1((A_2A_3)A_4))$: $100 \times 5 \times 50 + 100 \times 50 \times 30 + 10 \times 100 \times 30 = 205000$
- (3) $((A_1A_2)(A_3A_4))$: $10 \times 100 \times 5 + 5 \times 50 \times 30 + 10 \times 5 \times 30 = 14000$
- (4) $((A_1(A_2A_3))A_4)$: $100 \times 5 \times 50 + 10 \times 100 \times 50 + 10 \times 50 \times 30 = 90000$
- (5) $((((A_1A_2)A_3)A_4))$: $10 \times 100 \times 5 + 10 \times 5 \times 50 + 10 \times 50 \times 30 = 22500$



分治递归与指数瓶颈



代价计算公式的统一形式

无论采用何种算法，计算两个子矩阵链相乘的代价是其共同核心。

若将子链 $A_i \cdots A_j$ 在位置 k 处分割

通用状态转移方程

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + cost(i, j))$$

$$Cost = p_{i-1} \times p_k \times p_j$$

此乘积项只与 分割点 k 及其 左右两侧矩阵的行列 相关。

状态变量与目标函数设定

状态变量

$$m[i, j]$$

表示计算矩阵链 $A_i A_{i+1} \cdots A_j$ 所需的最少标量乘法次数。

$$s[i, j]$$

记录达到最优值 $m[i, j]$ 时的最优分割位置 k 。

目标函数

最终目标

求解 $m[1, n]$ ，即计算整个矩阵链 $A_1 \cdots A_n$ 的最小代价。

方案回溯

利用 s 表进行回溯，可以构造出具体的最优括号化方案。

分治法：思想与递归方程

算法思想

将矩阵链从位置 **k** 处分割，递归求解左右两个子链的最优解，然后合并。

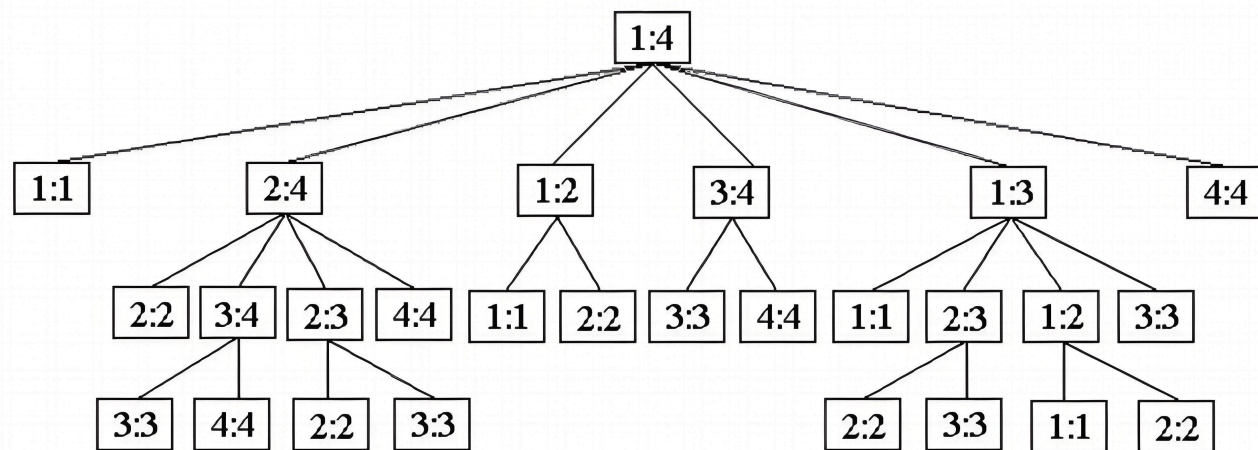
递归方程

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$$

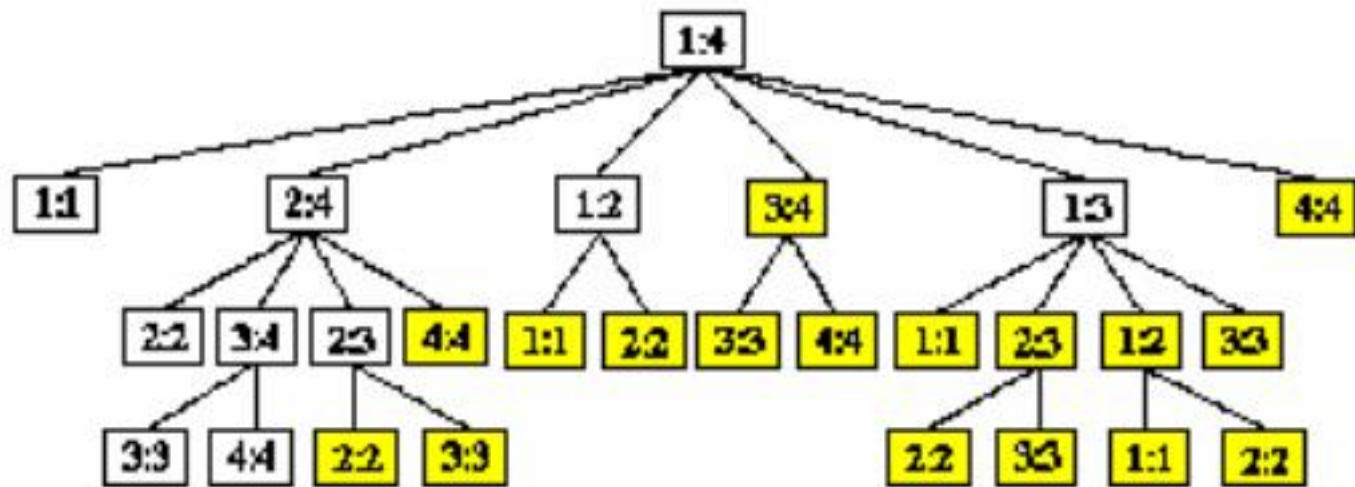
边界条件: $m[i, i] = 0$

```
int recurMatrixChain(int i, int j)
{
    if (i == j) return 0;
    int u = recurMatrixChain(i, i) + recurMatrixChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = recurMatrixChain(i, k) + recurMatrixChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) {
            u = t;
            s[i][j] = k;
        }
    }
    return u;
}
```

recurmatrixChain(1, 4)计算A[1:4]的递归树。



分治法的瓶颈：指数级复杂度



递归调用树分析

以 $n=4$ 为例，多个子问题被重复计算多次。
递归树深度为 $n-1$ ，节点总数呈指数级增长。

该递归算法的计算时间 $T(n)$ 可递归定义如下：

$$T(n) \geq \begin{cases} O(1) & n=1 \\ 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) & n > 1 \end{cases}$$

当 $n > 1$ 时，
$$T(n) \geq 1 + (n-1) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k) = n + 2 \sum_{k=1}^{n-1} T(k)$$

以上递归式，可用数学归纳法证明 $T(n) \geq 2^{n-1} = \Omega(2^n)$

时间复杂度: $O(2^n)$

空间复杂度: $O(n)$

*注：分治法仅具教学演示价值，无法处理大规模问题。

分治法：复杂度剖析与结论

$$O(2^n)$$

指数级时间

n=20时，调用次数 > 100万

VS

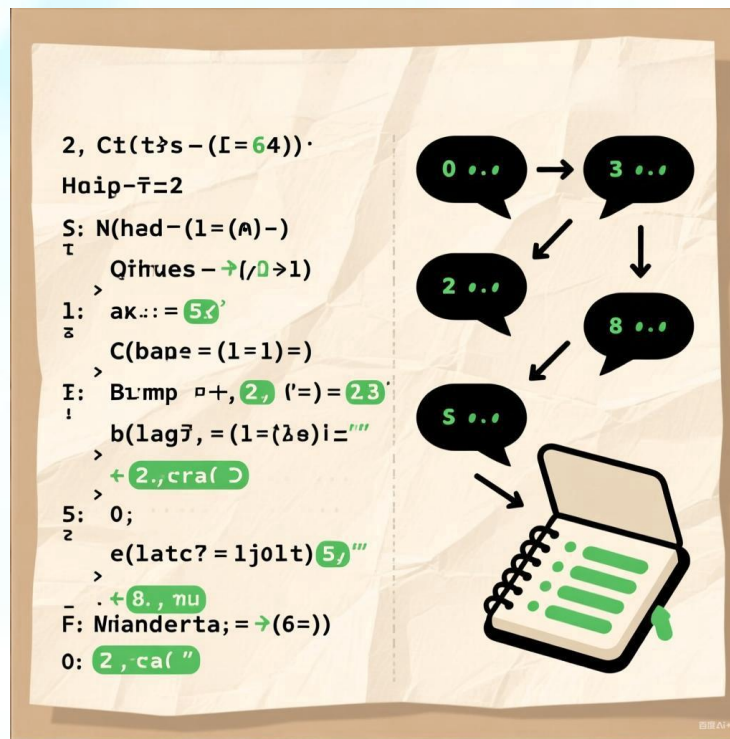
$$O(n^3)$$

多项式级时间

n=20时，计算次数 ≈ 2660

结论：分治法因大量重复计算导致指数级时间复杂度，在实际应用中不可行，必须引入动态规划进行优化。

PART 04



自顶向下备忘录

自顶向下：备忘录方法

在保持递归结构清晰性的同时，通过备忘录 (memo) 记录已计算结果，避免重复计算。

动态规划法：自底向上
备忘录方法：自顶向下

```
int LookupChain(int i, int j)
{
    if (m[i][j] > 0) return m[i][j];
    if (i == j) return 0;
    int u = LookupChain(i, i) + LookupChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = LookupChain(i, k) + LookupChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k;}
    }
    m[i][j] = u;
    return u;
}
```

初始化：将所有 $m[i,j]$ 初始化为 -1 (未计算)。

查询：计算前，先查询备忘录。若已计算 ($m[i,j] \neq -1$)，直接返回结果。

存储：计算完成后，将结果存入备忘录 $m[i,j]$ 。

优势：按需计算，可能跳过部分不必要的子问题。

自顶向下：复杂度与风险

复杂度

$O(n^3)$

每个子问题只计算一次，但递归调用有一定常数开销。

风险

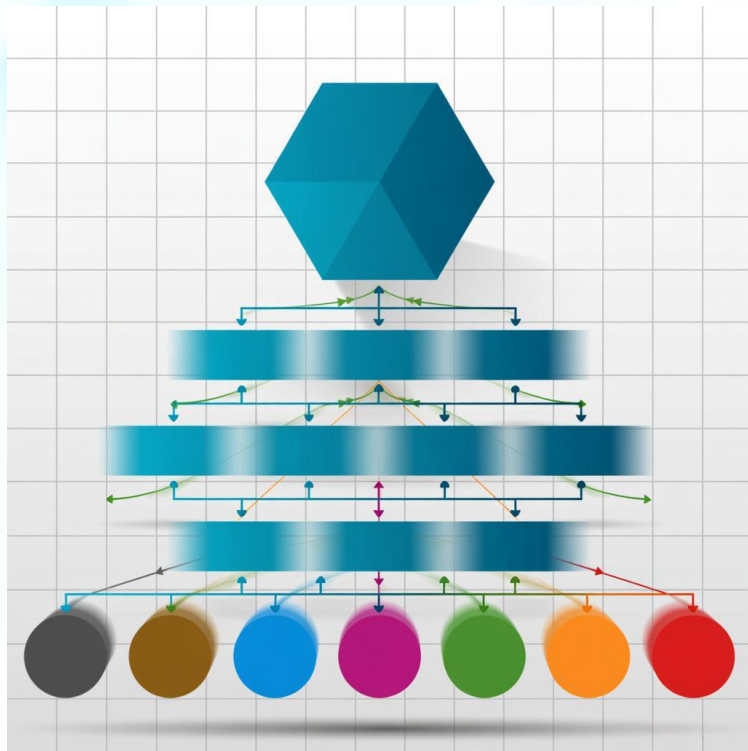
栈溢出

当 n 过大或平台栈深度受限时，可能发生运行时崩溃。

自顶向下方法在代码简洁性和思维直观性上有优势，但在工程应用中需注意栈空间限制。



PART 04



自底向上动态规划



动态规划基本步骤

将矩阵连乘积 $A_i A_{i+1} \cdots A_j$ 简记为 $A[i:j]$, 这里 $i \leq j$

考察计算 $A[i:j]$ 的最优计算次序。

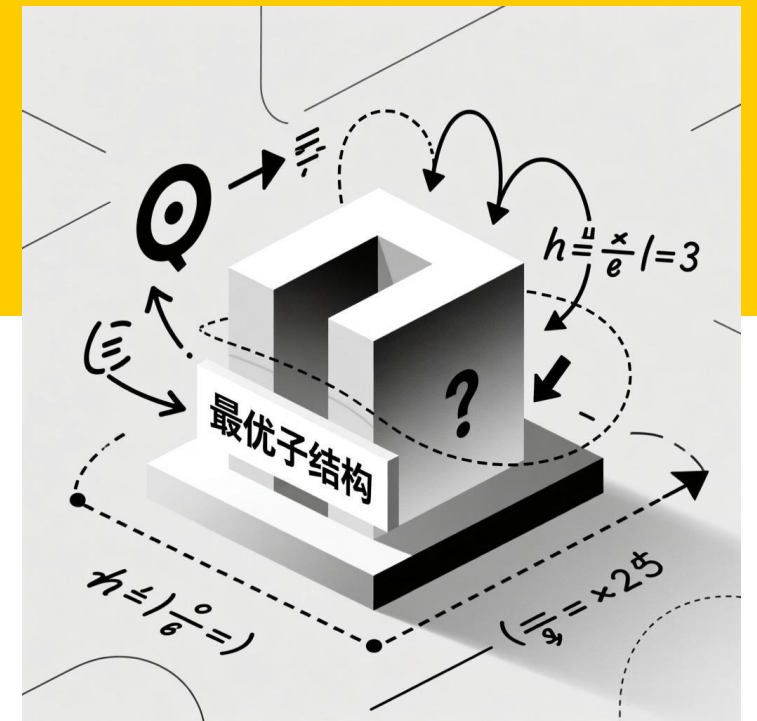
设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开, $i \leq k < j$, 则其相应完全加括号方式为 $(A_i A_{i+1} \cdots A_k)(A_{k+1} A_{k+2} \cdots A_j)$

计算量: $A[i:k]$ 的计算量加上 $A[k+1:j]$ 的计算量, 再加上 $A[i:k]$ 和 $A[k+1:j]$ 相乘的计算量



1. 分析最优解的结构

- 特征：计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。
- 如何证明？——反证法



2. 建立递归关系

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
- 当 $i=j$ 时, $A[i:j]=A_i$, 因此, $m[i,i]=0$, $i=1,2,\dots,n$
- 当 $i < j$ 时,

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

这里 A_i 的维数为 $p_{i-1} \times p_i$

- 由最优子结构性质, 当 $i \neq j$ 时, 可以递归地定义 $m[i,j]$ 为:

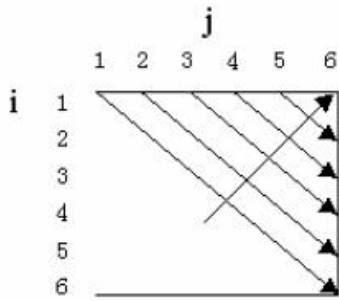
$$m[i][j] = \min_{i \leq k < j} \{m[i][k] + m[k + 1][j] + p_{i-1} \times p_k \times p_j\}$$

k 的位置只有 $j-i$ 种可能

3. 使用自底向上的方式解决重复计算问题

先解子问题，合并子问题，得到全解

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

	1	2	3	4	5	6
i 1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) $m[i][j]$

	1	2	3	4	5	6
i 1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) $s[i][j]$

- 1. 计算 $A_1, A_2, A_3, A_4, A_5, A_6$
- 2. 计算 $(A_1A_2), (A_2A_3), (A_3A_4), (A_4A_5), (A_5A_6)$
- 3. 计算 $(A_1A_2A_3)(A_4A_5A_6)$
- 4. 计算 $A[1:4], A[2:5], A[3:6]$
- 5. 计算 $A[1:5], A[2:6]$
- 6. 计算 $A[1:6]$

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & i < j \end{cases}$$

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

自底向上：填表顺序与策略

为确保计算当前状态时，其依赖的子问题已经解决，必须按链长 l 递增的顺序进行计算。

初始化：主对角线 $m[i,i] = 0$ 。

外层循环 (l)：控制链长，从 2 到 n 。

中层循环 (i)：控制起始位置。

内层循环 (k)：枚举分割点，计算最小代价。

用递推法方法实现

算法步骤：

```
1: for  $l = 2$  to  $n$ :           //  $l$  为子链长度
2:   for  $i = 0$  to  $n-l$ :
3:      $j \leftarrow i + l - 1$ 
4:      $m[i][j] \leftarrow \infty$ 
5:     for  $k = i$  to  $j-1$ :
6:        $q \leftarrow m[i][k] + m[k+1][j] + p[i]*p[k+1]*p[j+1]$ 
7:       if  $q < m[i][j]$ :
8:          $m[i][j] \leftarrow q$ 
9:          $s[i][j] \leftarrow k$ 
14: return  $(m, s)$ 
```

```
void MatrixChain(int *p, int n, int **m, int **s)
{
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
            s[i][j] = i;
            for (int k = i+1; k < j; k++) {
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
            }
        }
}
```

用动态规划法求最优解

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
{
```

```
    for (int i = 1; i <= n; i++) m[i][i] = 0;
```

```
    for (int r = 2; r <= n; r++) // r代表了对角线，如图a
```

```
        for (int i = 1; i <= n - r + 1; i++) { //代表了对每条对角线的数值
```

```
            int j=i+r-1;
```

```
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
```

```
            s[i][j] = i;
```

```
            for (int k = i+1; k < j; k++) {
```

```
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
```

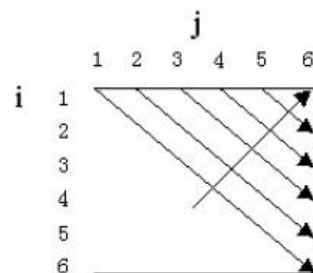
```
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k;}
```

```
            }
```

```
    }
```

```
}
```

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

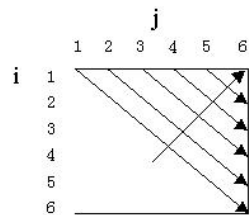
用动态规划法求最优解

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
{
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
            s[i][j] = i;
            for (int k = i+1; k < j; k++) {
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
            }
        }
}
```

- 对于 $1 \leq i \leq j \leq n$ 不同的有序对 (i, j) 对应于不同的子问题。因此，不同子问题的个数最多只有 $\binom{n}{2} + n = \Theta(n^2)$

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

	1	2	3	4	5	6
i 1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) $m[i][j]$

	1	2	3	4	5	6
i 1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) $s[i][j]$

算法复杂度分析：

算法matrixChain的主要计算量取决于算法中对r, i和k的3重循环。循环体内的计算量为 $O(1)$ ，而3重循环的总次数为 $O(n^3)$ 。因此，算法的计算时间上界为 $O(n^3)$ 。算法所占用的空间显然为 $O(n^2)$ 。

4. 构造最优值

■事实上，matrixChain已记录了构造最优解所需要的全部信息。 $s[i][j]$ 中的数 k 表明计算矩阵链 $A[i:j]$ 的最佳方式应在矩阵 A_k 和 A_{k+1} 之间断开，即最优的加括号方式应为 $(A[i:k])(A[k+1:j])$ 。

■因此，从 $s[1][n]$ 记录的信息可知计算 $A[1:n]$ 的最优加括号

方式为 $(A[1:s[1][n]])(A[s[1][n]+1:n])$ 。

■而 $A[1:s[1][n]]$ 的最优加括号方式为

$(A[1:s[1][s[1][n]]])(A[s[1][s[1][n]]+1:s[1][n]])$ 。同理可以确定 $A[s[1][n]+1:n]$ 的最优加括号方式在 $s[s[1][n]+1][n]$ 处断开，....,照此递推下去，最终可以确定 $A[1:n]$ 的最优完全加括号方式，即构造出问题的一个最优解。

```
void traceback (int i, int j, int ** s)
{
    if (i == j) return;
    traceback(s, i, s[i][j]);
    traceback(s, s[i][j] + 1, j);
    cout<<"Multiply A "<< i << ", " << s[i][j];
    cout<<" and A " << (s[i][j] + 1) << ", " << j << endl;
}
```

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

要输出 $A[1:n]$ 的最优计算次序只要调用 $\text{traceback}(1,n,s)$ 即可。对于上面所举的例子，通过调用 $\text{traceback}(1,6,s)$,可输出最优计算次序 $((A_1(A_2A_3))((A_4A_5)A_6))$

动态规划算法的基本要素

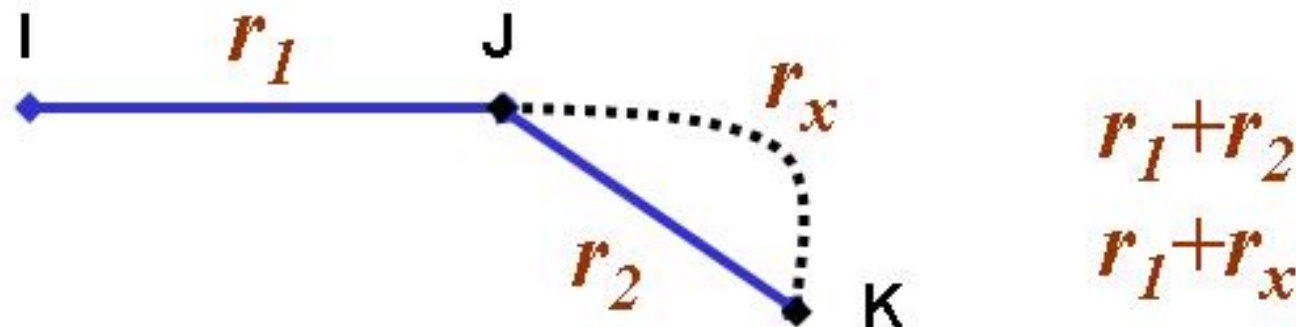
一、最优子结构

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。
- 在分析问题的最优子结构性质时，**所用的方法具有普遍性**：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，以**自底向上**的方式递归地从子问题的最优解逐步构造出整个问题的最优解。**最优子结构是问题能用动态规划算法求解的前提**。

同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）

想想看：

最优化原则，对于计算机网络中的分组转发有什么作用？



如果I, J, K的最佳路由是 $r_1 + r_2$ ，则当I把分组交给J时，J一定会沿着 r_2 把分组转发出去。

二、重叠子问题-自底向上的解决方法

-
- The diagram illustrates the prime factorization of the number 14. The root node is 14. It branches into 11, 2x4, 12, 3x4, 13, and 4x4. Further branches show the prime factorization of these intermediate products, such as 2x4 branching into 22, 34, 23, and 44, which then further branch into prime factors like 33, 44, 22, and 33.

自底向上：复杂度分析



时间复杂度: $O(n^3)$

三重循环 (链长 $\times$ 起始位置 $\times$ 分割点)。子问题总数 $O(n^2)$ ，每个需 $O(n)$ 时间计算。

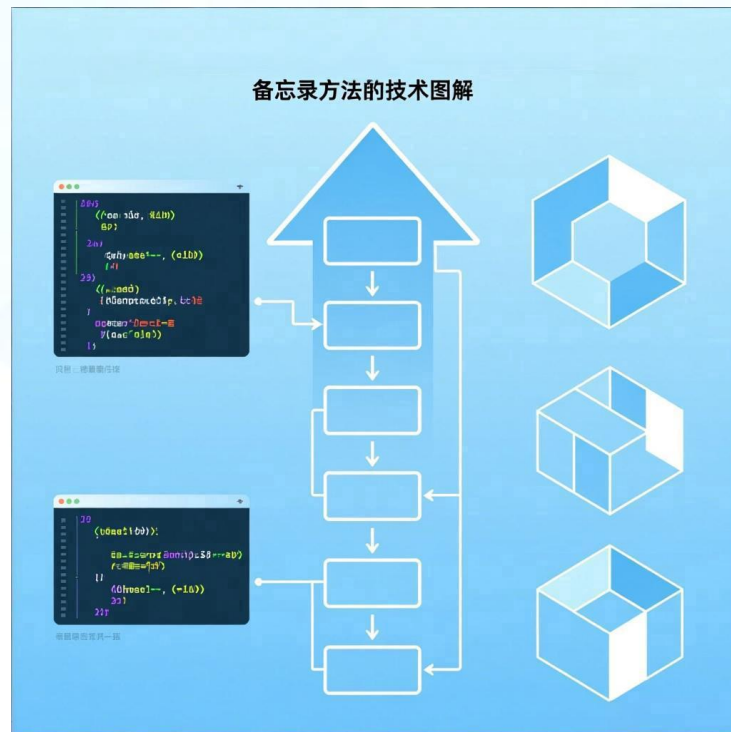


空间复杂度: $O(n^2)$

需要存储 m 表和 s 表，其大小均为 $n \times n$ 。

相比分治法，动态规划将时间复杂度从指数级降到了多项式级，效率提升巨大。

PART 05



三方法对比与选型

三方法全面对比

方法	时间复杂度	空间复杂度	优点	缺点
分治法	$O(2^n)$	$O(n)$	实现简单直观	指数时间，不可行
自底向上DP	$O(n^3)$	$O(n^2)$	无递归开销，效率稳定	需分析计算顺序
自顶向下备忘录	$O(n^3)$	$O(n^2)$	思路自然，按需计算	递归开销，栈溢出风险

工程落地与并行化展望

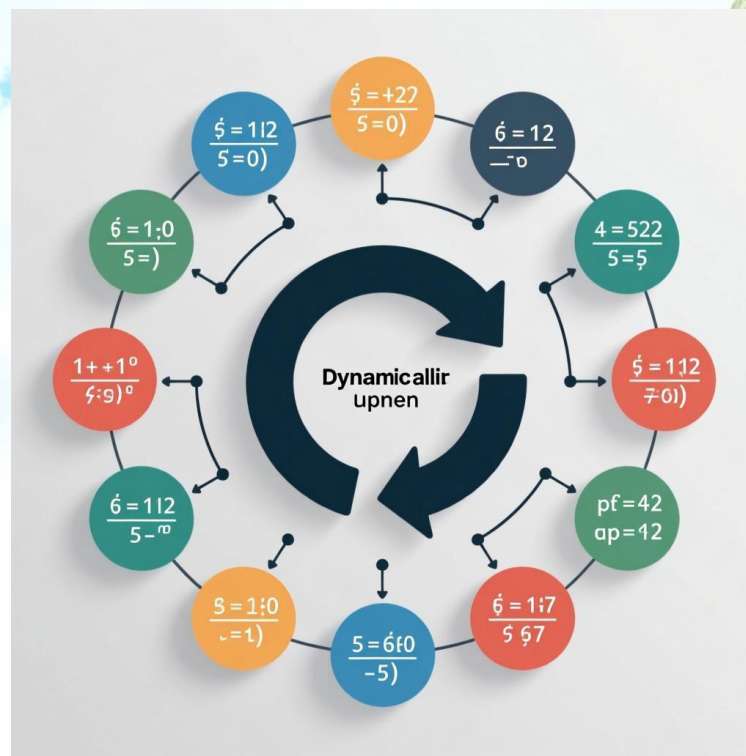
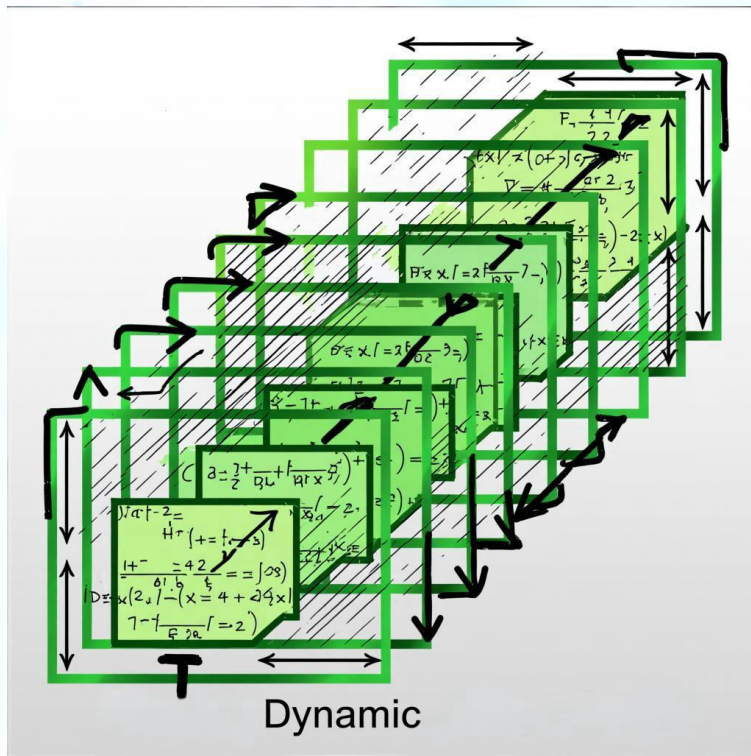
工程选型建议

小规模/原型：优先选择自顶向下，代码简洁。
大规模/生产：优先选择自底向上，性能稳定。
仅需值/需方案：若无需回溯方案，可考虑空间优化。

并行化潜力

自底向上：同链长 l 的各子问题可并行计算。
挑战：负载均衡，当矩阵维度差异巨大时，子问题计算量不均。
前沿：研究基于块循环的分布式DP算法。

PART 06



实战优化

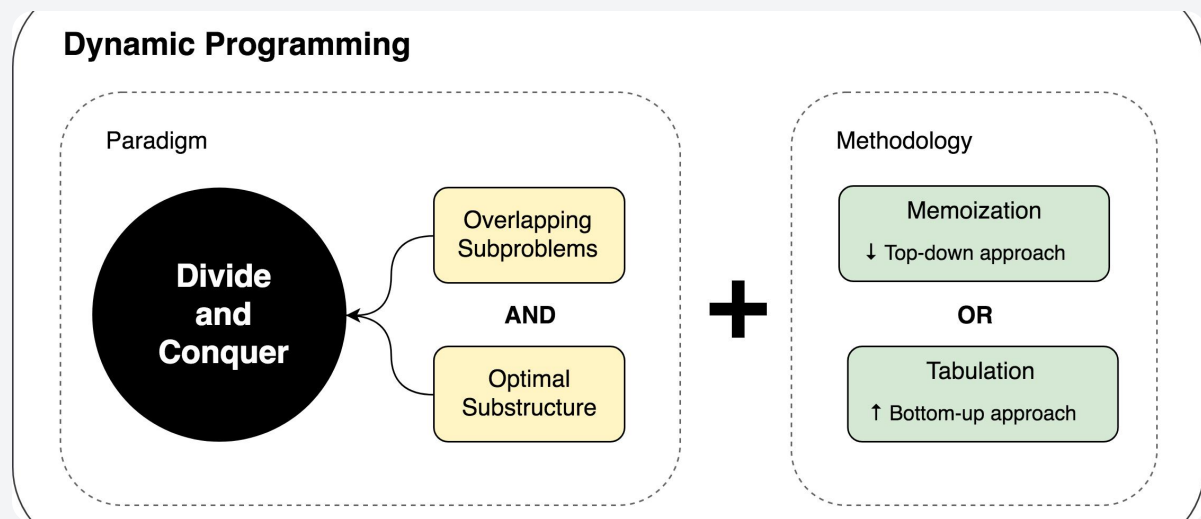
拓展：空间优化与滚动数组

当仅需计算最小代价，而不需要回溯具体方案时，可以进行空间优化。

滚动数组技巧

由于计算 $m[i,j]$ 时，只需要用到 **本层** 和 **下一层** 的数据，因此可以将 $O(n^2)$ 的 m 表优化至 $O(n)$ 。

*注意：此优化会丢失回溯信息，无法构造最优方案。



拓展：维度差异与启发式分割

当矩阵维度差异巨大时，固定枚举所有 k 会造成大量无效计算。

启发式策略

可以基于维度乘积 $p[k-1]*p[k]*p[k+1]$ 等阈值进行**剪枝**，或利用机器学习预测近似最优的 k 值范围，减少搜索空间。

Case – TSP – 禁忌搜

- 位置4与位置6上的元素交换后，位置4上的元素在之后的6步移动中都不允许再次发生改变。
- 上图中加下划线的地方。
- 这样的策略既可以保持相对丰富的可用移动，又能防止搜索方向的立即逆转。

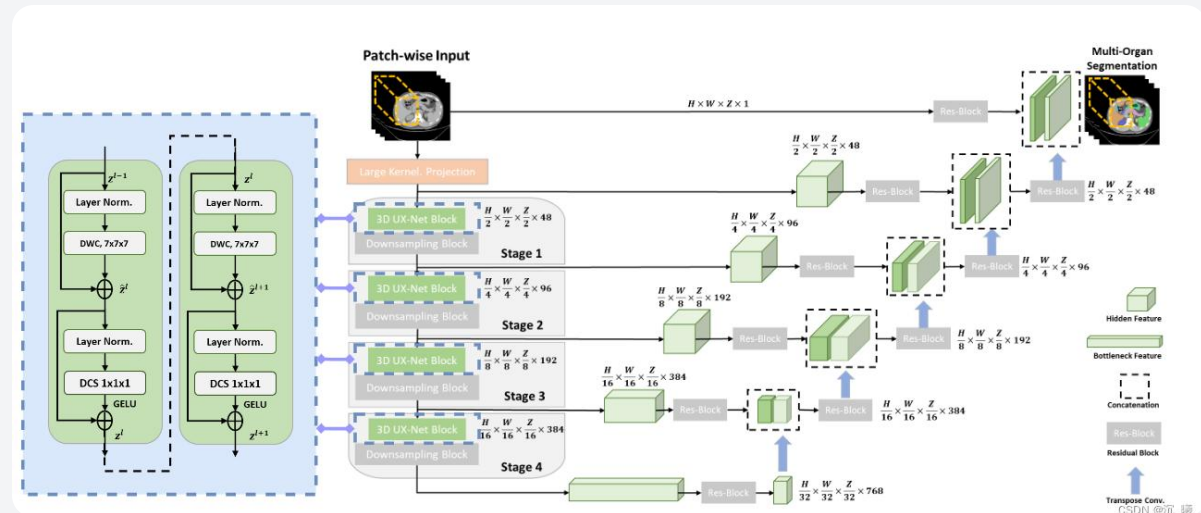
1	1-2-3-6-5-4-7-8-9-10	99.6	99.6	第1个位置和第2个位置交换	7.1
2	2-1-3-6-5-4-7-8-9-10	97.7	97.7	第8个位置和第10个位置交换	1.9
3	2-1-3-6-5-4-7-10-9-8	96.0	96.0	第7个位置和第10个位置交换	2.2
4	2-1-3-6-5-4-8-10-9-7	93.8	93.8	第6个位置和第10个位置交换	2.2
5	2-1-3-6-5-7-8-10-9-4	96.1	93.8	第5个位置和第10个位置交换	0.1
6	2-1-3-6-4-7-8-10-9-5	96.2	93.8	第4个位置和第10个位置交换	2.9
7	2-1-3-5-4-7-8-10-9-6	93.3	93.3	第1个位置和第3个位置交换	1.6
8	3-1-2-5-4-7-8-10-9-6	91.7	91.7	第8个位置和第9个位置交换	0.2
9	3-1-2-5-4-7-8-9-10-6	91.9	91.7	第2个位置和第3个位置交换	1.7
10	3-2-1-5-4-7-8-9-10-6	93.6	91.7	第6个位置和第7个位置交换	3.2
11	3-2-1-5-4-8-7-9-10-6	96.8	91.7	第5个位置和第7个位置交换	3.0
12	3-2-1-5-7-8-4-9-10-6	99.8	91.7	第4个位置和第7个位置交换	15.9
13	3-2-1-4-7-8-5-9-10-6	83.9	83.9	第1个位置和第3个位置交换	2.1
14	1-2-3-4-7-8-5-9-10-6	86.0	83.9	第8个位置和第9个位置交换	0.5
15	1-2-3-4-7-8-5-10-9-6	86.5	83.9	第2个位置和第3个位置交换	0.8
:	:	:	:	:	:
40	1-2-7-8-5-9-10-4-6-3	96.3	83.9	第1个位置和第2个位置交换	1.3
41	2-1-7-8-5-9-10-4-6-3	97.6	83.9	第8个位置和第9个位置交换	9.9
42	2-1-7-8-5-9-10-6-4-3	87.7	83.9	第2个位置和第9个位置交换	0.9
43	2-4-7-8-5-9-10-6-1-3	86.8	83.9	第9个位置和第10个位置交换	5.0
44	2-4-7-8-5-9-10-6-3-1	81.8	81.8	第6个位置和第7个位置交换	0.5
45	2-4-7-8-5-10-9-6-3-1	82.3	81.8	第5个位置和第7个位置交换	3.1
46	2-4-7-8-9-10-5-6-3-1	85.4	81.8	第1个位置和第10个位置交换	2.1
47	1-4-7-8-9-10-5-6-3-2	87.5	81.8	第7个位置和第8个位置交换	0.7

前沿：并行化与分布式填表

对于超大规模问题($n > 10^4$)，单机计算时间过长，需借助并行计算。

挑战与前沿

传统的按反斜线并行策略可扩展性有限。前沿研究聚焦于通信避免算法和基于块循环的分布式DP，以在超级计算机上实现更高效率。





PART 07



课程总结与思考



课程总结：关键知识点回顾



判定

矩阵连乘具备最优子结构与重叠子问题。



顺序

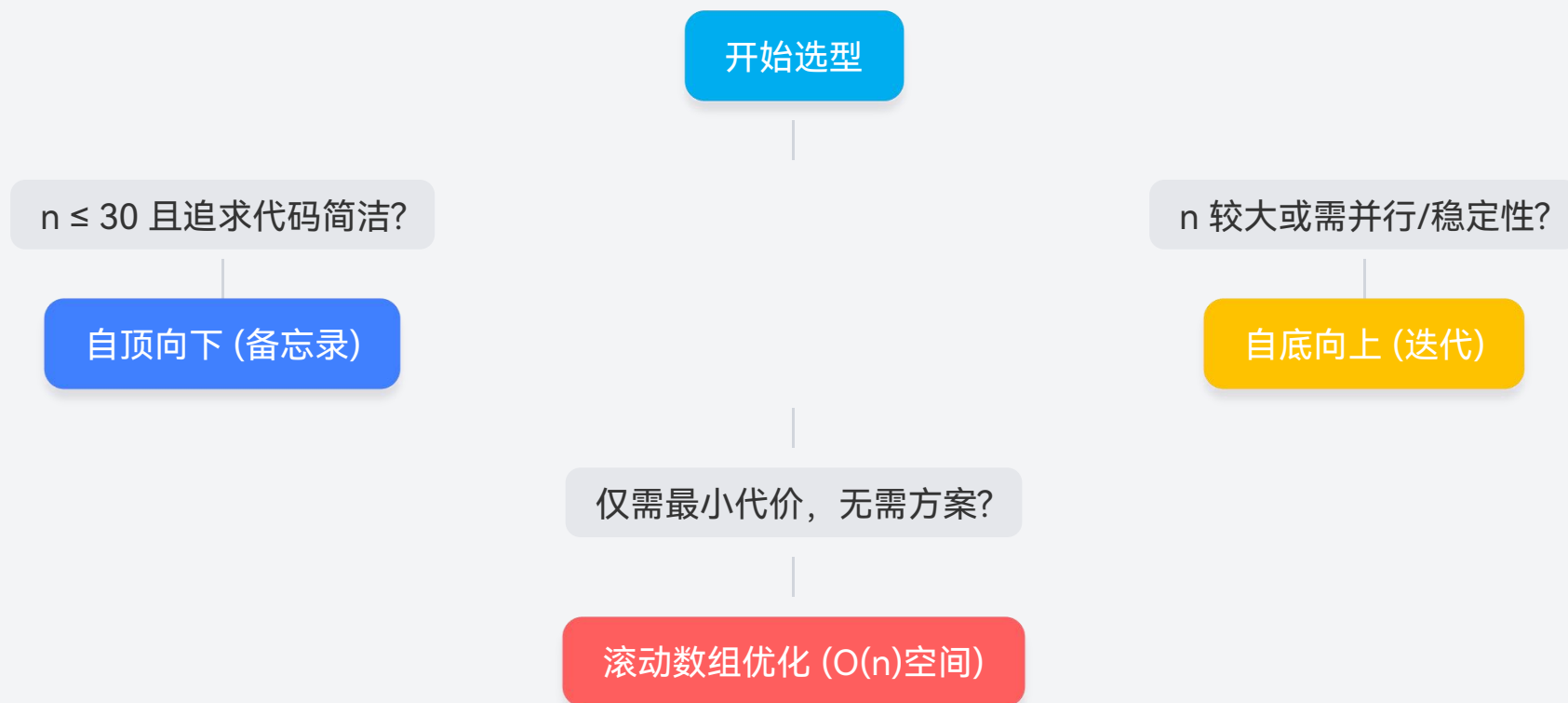
自底向上的填表顺序是正确性的根基。



回溯

保存s表即可回溯构造最优方案。

算法选型决策树



开放性问题与未来方向



如何证明矩阵连乘不存在 $o(n^3)$ 的精确算法？



若允许近似最优，能否实现 $O(n \log n)$ 的亚立方解法？



在量子计算模型下，DP填表能否获得二次加速？



THANKS!

石子合并：有 n 堆石子排成一排，每堆石子有一定的数量，将 n 堆石子合并成一堆。

合并规则：每次只能合并相邻的两堆石子，合并的花费为这两堆石子的总数。石子经过 $n-1$ 次合并后成为一堆，求总的最小花费。

